

Manual Técnico — 3xtrat3rr3str3D

Compilador de tres lenguajes de alto nivel (**Lat** / **Pig Latin**, **Y?** y **Zetariano**) hacia **código de tres direcciones (C3D)** y, finalmente, a un programa **C** compilable con gcc.

Proyecto	Compi2 — Proyecto 1: C3D ejecutable
Artefacto	com.navi:3xtrat3rr3str3D:1.0-SNAPSHOT
Lenguaje de implementación	Java 17
Herramienta de análisis	ANTLR 4.13.2
Build	Maven (3.9.x)
Fecha de documentación	2026

1. Introducción

1.1 Objetivo

Construir un compilador que reciba programas escritos en tres lenguajes de alto nivel de la Resistencia y:

1. Valide la sintaxis y la semántica (léxica, sintáctica y semánticamente).
2. Genere **cuartetos** de código de tres direcciones.
3. Traduzca esas cuartetos a un **programa C autocontenido**, compilable con gcc, que ponga en práctica los conceptos de **stack** y **heap**.

1.2 Alcance

- Soporte para los tres lenguajes descritos en el enunciado.
- **Lat** es el lenguaje de entrada principal y el único que puede **importar** módulos .y y .z. Y y Z no importan entre sí.
- Interfaz gráfica para edición, resaltado en tiempo real, gestión del árbol de trabajo y ejecución del pipeline de compilación.
- La herencia, el encapsulamiento y el polimorfismo en Z se dejan para el Proyecto 2 (el diseño está preparado para escalar).

1.3 Pipeline de compilación

flowchart LR

A[Fuente .pig/.y/.z] --> B[Lexer ANTLR]

B --> C[Parser ANTLR]

C --> D[AST del lenguaje]

D --> E[Análisis semántico]

E --> F[Tabla de símbolos + tipos]

F --> G[Generador C3D]

G --> H[Cuartetas]

H --> I[CGenerator]

I --> J[archivo .c]

J --> K[gcc]

K --> L[ejecutable]

2. Tecnologías utilizadas

Tecnología	Versión	RoI
Java	17	Lenguaje de implementación (records, switch expressions, pattern matching)
ANTLR	4.13.2	Generación de lexers, parsers y visitors
Maven	3.9.x	Gestión de build y dependencias

Tecnología	Versión	Rol
Lombok	1.18.46	@Getter/@Setter para reducir boilerplate
FlatLaf	3.2.1	Look & feel moderno para Swing (tema Material Darker)
flatlaf-intellij-themes	3.2.1	Tema FlatMaterialDarkerIJTheme
Swing	(JDK)	Interfaz gráfica (editor, consola, paneles, explorador)
gcc	(sistema)	Compilación del C generado

2.1 Dependencias del pom.xml

```
<dependency>
  <groupId>org.antlr</groupId>
  <artifactId>antlr4</artifactId>
  <version>4.13.2</version>
</dependency>
```

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.46</version>
</dependency>
```

```
<dependency>
  <groupId>com.formdev</groupId>
  <artifactId>flatlaf</artifactId>
  <version>3.2.1</version>
</dependency>
```

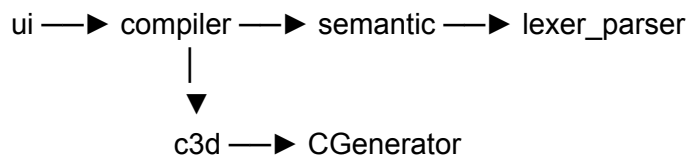
```
<dependency>
  <groupId>com.formdev</groupId>
  <artifactId>flatlaf-intellij-themes</artifactId>
  <version>3.2.1</version>
</dependency>
```

El plugin `antlr4-maven-plugin` 4.13.2 se ejecuta con `visitor=true`.

Decisión de diseño: no se usa ninguna librería de editor de texto ni de resaltado (por ejemplo `RSyntaxTextArea`). El resaltado se implementa con los propios lexers ANTLR y `StyledDocument` de Swing (ver §9.3).

3. Arquitectura general

El sistema se organiza en capas con dependencias unidireccionales:



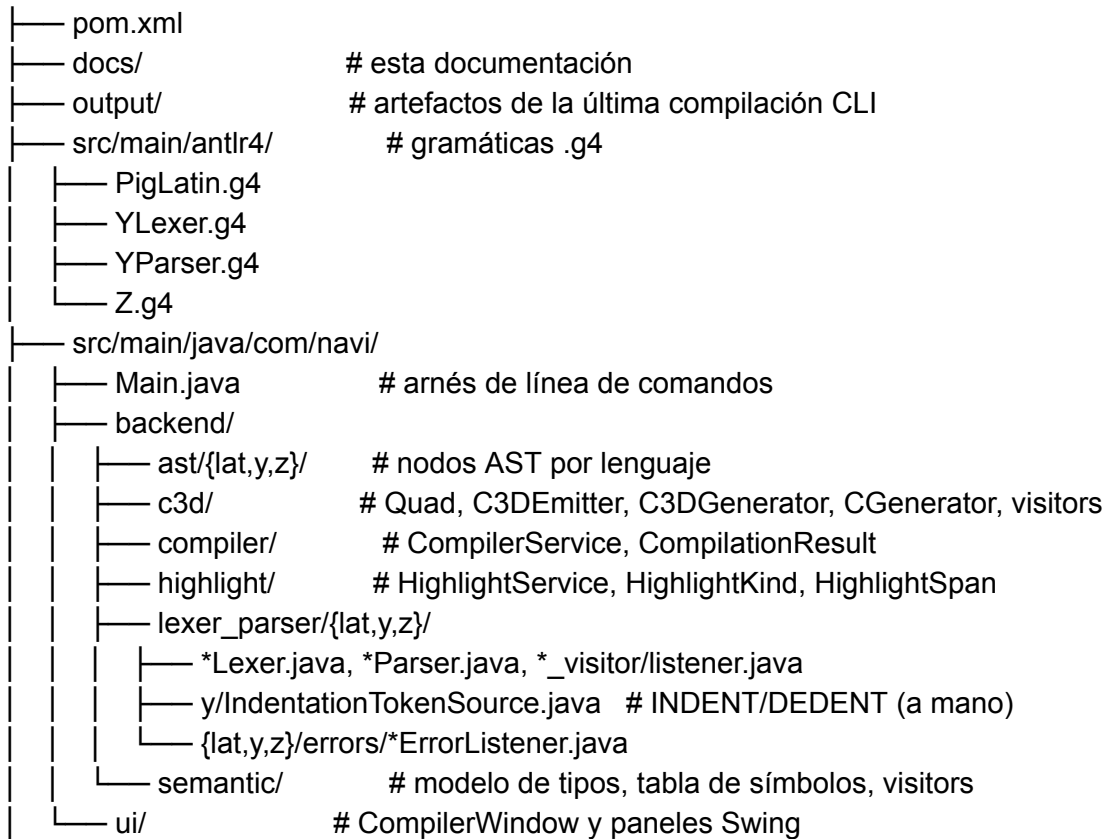
Capa	Paquete	Responsabilidad
Presentación	<code>com.navi.ui</code> , <code>com.navi.backend.highlight</code>	Interfaz, resaltado, explorador, paneles de resultados
Orquestación	<code>com.navi.backend.compiler</code>	Ejecuta el pipeline completo y produce el C
Semántica	<code>com.navi.backend.semantic</code>	Validación, tabla de símbolos, tipos, imports
AST	<code>com.navi.backend.ast.{lat,y,z}</code>	Modelo de datos de cada lenguaje
C3D	<code>com.navi.backend.c3d</code>	Generación de cuartetos y traducción a C
Generado	<code>com.navi.backend.lexer_parser.{lat,y,z}</code>	Lexers/parsers ANTLR (copias versionadas)

3.1 Entradas del sistema

Entrada principal en Lat	Módulos importados	Salida
archivo.pig	.y, .z	archivo.c + ejecutable en output/
archivo.y	—	archivo.c + ejecutable
archivo.z	—	archivo.c + ejecutable

4. Estructura del proyecto

3xtrat3rr3str3D/



4.1 El "gotcha" de la regeneración de gramáticas

`mvn compile` regenera las clases ANTLR en el **paquete por defecto** dentro de `target/generated-sources/antlr4`, pero el código importa **copias versionadas** bajo `com.navi.backend.lexer_parser.<lang>`. Por tanto, tras editar un .g4:

1. Ejecutar `mvn generate-sources`.
2. Copiar los .java generados a `src/main/java/com/navi/backend/lexer_parser/<lat|y|z>/`.
3. Anteponer `package com.navi.backend.lexer_parser.<lang>;` en la primera línea.

No sobrescribir los archivos escritos a mano:

- `lexer_parser/y/IndentationTokenSource.java` (INDENT/DEDENT de Y).
- `lexer_parser/{lat,y,z}/errors/*ErrorListener.java`.

5. Análisis léxico y sintáctico

5.1 Gramáticas

Las gramáticas completas están en el **Anexo A**. Resumen:

Lenguaje	Gramática	Particularidad
Lat	PigLatin.g4 (combinada)	Secciones con tokens VARIABLES>, MUNERA>, MAIOR>; comentarios ## ... ##
Y	YLexer.g4 + YParser.g4 (separada)	Indentación significativa (INDENT/DEDENT)
Z	Z.g4 (combinada)	Orientado a objetos; una clase por archivo

5.2 Indentación en Y

ANTLR no produce token de indentación de forma nativa. La clase escrita a mano `IndentationTokenSource` envuelve al `YLexer` y emite tokens `INDENT/DEDENT` comparando la columna de inicio de cada línea con una pila de indentaciones. El parser recibe esos tokens para delimitar bloques sin llaves.

5.3 Construcción del AST

Cada lenguaje tiene un visitor propio que recorre el árbol de parseo y construye nodos del AST (`com.navi.backend.ast.<lang>`):

- `FileModuleLoader.parseLat/parseY/parseZ` crean el lexer y parser, instalan un `CollectingErrorListener` y construyen el `Program`, `ProgramY` o `ProgramZ`.
- Si el listener registró errores, se lanza `SemanticError` con el texto de los mismos y la compilación se detiene antes de construir el AST.

5.4 Errores léxicos y sintácticos

`CollectingErrorListener` extiende `BaseErrorListener` y acumula los errores en una lista (en vez de imprimirlos), con el formato `[line:col] mensaje`. ANTLR entrega tanto errores léxicos como sintácticos por `syntaxError`, por lo que no se distinguen en la lista.

Además existen `YErrorListener` y `ZErrorListener` (listeners "clásicos" que imprimen a `stderr`) que quedan como utilidades heredadas; el pipeline actual usa `CollectingErrorListener`.

5.5 Resaltado en tiempo real

HighlightService reutiliza los lexers ANTLR para obtener los tokens y clasificar cada uno en un HighlightKind. No usa regex ni librerías. El editor aplica los HighlightSpan sobre un StyledDocument (ver §9.3).

6. Análisis semántico

La capa semántica vive en `com.navi.backend.semantic`. Su meta es **validar** y **anotar** el AST para alimentar al generador de C3D, manteniendo un modelo canónico e independiente del lenguaje.

6.1 Modelo canónico

6.1.1 Tipos (Type)

Type es el único sistema de tipos: VOID, BOOLEAN, INT, DOUBLE, CHAR, STRING (singletons), ARRAY(elementType, dimensions), STRUCT(name), CLASS(name), NULL y ERROR. Los tres lenguajes mapean sus palabras clave a estas mismas instancias (por ejemplo `numerus/entero/int` → `Type.INT`).

6.1.2 Agregados (AggregateType / Field)

AggregateType describe el *layout* de una estructura de Y o una clase de Z: lista ordenada de Field más un memberScope con métodos y constructores. Calcula fieldOffset, size y resolución por firma (findMethod, findConstructor).

6.1.3 Tabla de símbolos (SymbolTable / Scope)

Árbol de ámbitos léxicos con tipos GLOBAL, FUNCTION, CLASS, STRUCT, BLOCK, LOOP. Cada Scope guarda Map<String, List<Symbol>>, de modo que los **callable sobrecargan**. defineUnique se usa para variables/parámetros y defineCallable para funciones/métodos/constructores (rechaza mismo nombre y mismos tipos de parámetros).

Symbol contiene nombre, SymbolKind (VARIABLE/PARAMETER/FUNCTION/METHOD/CONSTRUCTOR), tipo, FunctionSignature, flag de referencia, ámbito, agregado propietario y posMemory (offset asignado luego por el emisor C3D; -1 hasta entonces).

6.1.4 Compatibilidad de tipos

TypeCompat define el ensanchamiento `CHAR` → `INT` → `DOUBLE`, la asignación (canAssign) y la comparabilidad (comparable). TypeRules centraliza la validación de operadores, asignaciones, retornos y accesos, y la redacción de mensajes. La tabla completa está en el **Anexo B**.

6.2 Contexto de análisis (SemanticContext)

Campo	Tipo	Propósito
symbolTable	SymbolTable	Símbolos globales de Lat + importados de Y/Z
typeTable	TypeTable	Layouts de struct/clase por nombre
loadedModules	Set<String>	Rutas ya cargadas (evita recarga/ciclos)
loadedAsts	Map<String, Object>	AST de cada módulo importado (para C3D)
annotations	Map<Object, Type>	Tipo resuelto de cada expresión (IdentityHashMap)
symbolBindings	Map<Object, Symbol>	Liga nodos de declaración con su Symbol
errors	SemanticErrors	Recolector de errores

annotate, bindSymbol, typeOf y symbolOf son los puentes hacia el C3D.

6.3 Estrategia de llenado (dos fases, imports primero)

`SemanticAnalyzer.analyze(lat Program):`

1. **Imports primero, en orden, eager.** Por cada `ImportDeclaration`: detectar el lenguaje por la extensión → `ModuleLoader` → visitor de declaraciones + visitor semántico sobre el AST cargado.
2. **Declaraciones de Lat:** `LatDeclarationVisitor` registra variables globales y firmas de funciones (permite recursión y llamadas a lo importado).
3. **Semántica de Lat:** `LatSemanticVisitor` valida y anota cuerpos y main.

Y usa tres sub-pasos en `YDeclarationVisitor.visit(ProgramY)` para permitir que los campos de una estructura referencien estructuras declaradas más abajo:

1. Registrar **nombres** de estructuras.
2. Registrar **firmas** de funciones.
3. Rellenar los **campos** de las estructuras.

6.4 Imports

- Solo **Lat** importa; Y y Z carecen de imports. El grafo de dependencias tiene profundidad ≤ 2 ($Lat \rightarrow \{Y, Z\}$), sin ciclos reales.
- Los imports son **import * planos**: los símbolos importados quedan disponibles por nombre simple en el **único ámbito global**. No hay acceso cualificado `modulo.símbolo`.
- La **última extensión** de la ruta determina el lenguaje ($y \rightarrow Y, z \rightarrow Z$). Cualquier otra extensión es error semántico.
- `FileModuleLoader` mapea los puntos a / y resuelve relativo al directorio del archivo .pig. Ejemplo: `import utils.utils.y` con el .pig en prueba/ resuelve a `prueba/utils/utils.y`.

6.5 Visitors y checkers

Cada lenguaje tiene:

- Un **visitor de declaraciones** (`AstXVisitor<Void>`): registra símbolos.
- Un **visitor semántico** (`AstXVisitor<Type>`): es un dispatcher delgado.
- Un **XStatementChecker** y un **XExpressionChecker** con la lógica real.
- Un **XTypeResolver** que mapea palabras de tipo a `Type`.

Los visitors semánticos devuelven el `Type` de cada expresión y llaman a `context.annotate(nodo, tipo)`. Las declaraciones se ligan a su `Symbol` vía `bindSymbol`.

6.6 Reglas de negocio destacadas

- **Recursividad y referencias adelantadas**: las firmas se registran antes de analizar cuerpos.
- **Sobrecarga**: métodos y constructores pueden compartir nombre si difieren en los tipos de sus parámetros.
- **this implícito**: el acceso a campos/métodos sin calificar dentro de una clase se resuelve con el `currentClass` del checker de Z. En C3D, `this` es el parámetro 0.
- **Constructores**: `novus/new` exige que exista un constructor para la lista de argumentos. No se sintetiza constructor por defecto.
- **Unicidad**: variables/parámetros únicos por ámbito; structs/clases únicos en la `TypeTable` plana; callables únicos por nombre + firma.
- **Errores acumulativos**: el análisis no aborta ante error semántico (se recolectan todos); solo el parseo/carga aborta la corrida.

7. Generación de C3D

El paquete `com.navi.backend.c3d` genera cuartetos a partir del AST, **separado** de la capa semántica (solo lee `SemanticContext`). El diseño sigue el modelo clásico de cuartetos con direccionamiento por Base Pointer y control de flujo por saltos explícitos. El detalle del vocabulario y del modelo de memoria está en el **Anexo E**.

7.1 Componentes

Clase	Rol
Quad	Cuarteto (op, result, args...) y su render textual
C3DEmitter	Acumula cuartetos, asigna temporales/labels y modela stack/heap
C3DGenerator	Orquesta: emite módulos importados y luego Lat (funciones + main)
LatC3DVisitor / YC3DVisitor / ZC3DVisitor	Visitor por lenguaje (AstXVisitor<String>)

Cada visitor se apoya en clases auxiliares `Resolver`, `Declaration`, `Statement` y `Expression` para mantener la lógica organizada.

7.2 Modelo de stack y heap

- **Stack:** cada función abre un marco con `enter <label>, <size>` y lo cierra con `leave` (emitido junto a cada `return`). Parámetros y locales se direccionan con el Base Pointer y offsets 0-based secuenciales: `t = BP + off; t2 = stack[t]`. Los globales de Lat usan GP: `t = GP + off; t2 = stack[t]`.
- **Heap:** modelado con HP. `new` reserva celdas (`t = HP; HP = HP + size`). Objetos Z y arreglos Z se acceden como `t = base + off; heap[t]`. Los structs Y viven en el stack.

7.3 Booleanos y control de flujo

Las comparaciones y lógicos se **materializan a 0/1 con saltos** y las bifurcaciones usan `if a op b goto L` (nunca `if_false`). `&&` y `||` hacen cortocircuito.

7.4 Código inalcanzable y estilo de saltos

Tras `return/goto/halt` se descartan cuartetas hasta la próxima etiqueta alcanzable. `finalQuads()` aplica después:

1. **Colapso de etiquetas vacías** (reubica los saltos a la etiqueta siguiente).
2. **Inserción de saltos explícitos** antes de cada etiqueta para no depender del *fall-through*.

7.5 Convención de etiquetas

Elemento	Etiqueta
Función	nombre
Método	Owner_nombre_tipoParam1_tipoParam2...
Constructor	Owner_init_tipoParam1...

Las llamadas pasan el receptor (`obj/this`) como primer argumento.

8. Generación de código C

CGenerator traduce la **lista final de cuartetas** (`C3DGenerator.quads()`) a un único archivo C autocontenido. El patrón es cuartetas → sentencias C directas.

8.1 Modos de emisión

- **Modo int:** se usa cuando ninguna cuarteta usa cadenas, caracteres, decimales, `null` ni `read`. Todos los valores son `long long`; operadores C nativos; `printf("%lld", ...)`; `stack[addr]` y `heap[addr]`.
- **Modo Val:** en caso contrario, un valor etiquetado `Val` con unión (`int/double/char/string/null`) y solo las ayudas que las cuartetas usan (`as_str`, `add`, `op`, `cmp`, `prn`, `rd`). Constructores `I/D/CH/S/NIL`; `IDX(v)` extrae dirección y `AS_D(v)` promueve a `double`.

8.2 Estructura del C

- Cada enter `<label>`, `<size>` se convierte en `static void __fn_<label>(void)`.
- Los temporales son variables locales C, por lo que la **recursión** funciona.

- `leave` restaura BP (`SP = BP; BP = __oldBP;`).
- `main` inicializa GP, HP, BP, SP con el tamaño global; llama a `globals_init` y `main` si existen.

8.3 Salida y limpieza

`CompilerService` y `Main` escriben el `.c` en `output/` **junto al archivo fuente**, limpian la carpeta de la corrida anterior y compilan con `gcc -O0 -o <base> <base>.c`. El ejecutable resultante se llama como el fuente sin extensión. Si `gcc` falla, se reporta el error con su salida.

9. Interfaz gráfica

La interfaz está implementada en Swing, bajo `com.navi.ui`. El look & feel es FlatLaf (tema Material Darker) con una paleta personalizada "Miku".

Nota de arranque: en `Main.java` el arranque de la GUI está actualmente **comentado**; el `main` activo es el arnés de línea de comandos (ver §13). Para usar la ventana, descomenta el bloque `main` alternativo en `Main.java` que crea `CompilerWindow` y aplica `FlatMaterialDarkerIJTheme.setup()`.

9.1 Composición de `CompilerWindow`

- Menú **Archivo**: Abrir proyecto, Guardar (Ctrl+S), Salir.
- Menú **Compilar**: Compilar archivo actual.
- Barra de herramientas: Compilar, Guardar, Limpiar consola.
- Split horizontal: explorador de proyecto (izquierda) / editor-resultados (derecha).
- Split vertical: pestañas de editor (arriba) / pestañas de resultados (abajo).
- Pestañas de resultados: **Consola**, **Tabla de símbolos**, **Tabla de tipos**, **C3D**.
- Barra de estado: línea, columna, nombre de archivo y codificación (UTF-8).

9.2 Editor (`EditorPanel`)

- `JTextPane` con números de línea (`NumeroLinea`), fuente monoespaciada 15.
- **Deshacer/rehacer** (`UndoManager`): Ctrl+Z, Ctrl+Shift+Z, Ctrl+Y.
- **Tab** inserta 4 espacios.
- Un `EditorPanel` por pestaña; `CompilerWindow` los gestiona con `JTabbedPane` (una pestaña por archivo). Click central cierra; click derecho ofrece cerrar todas / las demás.

9.3 Resaltado en tiempo real

HighlightService usa los **lexers ANTLR** para tokenizar el texto y clasificar cada token en HighlightKind:

HighlightKind	Ejemplos
KEYWORD	si, dum, definir, class, return
TYPE	numerus, entero, int, String
BOOLEAN	verum, falsus, true, false
NUMBER	10, 3.14
STRING	"texto"
CHAR	'a'
COMMENT	// ..., ## ... ##, /* ... */
OPERATOR	+, ==, &&, ++
PUNCTUATION	(,), {, ;, ,
IDENTIFIER	nombres de variables/funciones

El editor aplica los HighlightSpan como AttributeSet sobre un StyledDocument. CompilerWindow usa un Timer de 200 ms con setRepeats(false) y ejecuta el resaltado en un SwingWorker (fuera del EDT), descartando resultados obsoletos mediante un contador de generación.

9.4 Explorador de proyecto

- Árbol (JTree) con carpetas primero y orden alfabético.
- Oculta .git, .idea y target; muestra solo .pig, .y, .z (y todo dentro de output/, incluido el .c y el ejecutable).
- Doble click abre; click derecho ofrece **Nuevo archivo**, **Nueva carpeta** y **Eliminar**.
- ProjectFileService valida nombres y evita path traversal.

9.5 Consola y paneles de resultados

- `ConsolePanel`: salida con estilos normal/éxito/error y autoscroll.
- `SymbolTablePanel`: columnas Nombre, Kind, Tipo, Ámbito, Pos. memoria, Línea, Columna.
- `TypeTablePanel`: columnas Agregado, Categoría, Miembro, Tipo, Detalle.
- `C3DPanel`: cuartetos con números de línea.

9.6 Compilación desde la UI

`CompilerWindow.compile()` guarda el archivo, limpia la consola y llama a `CompilerService.compile(currentFile)`. Si hay errores, los vuelca a la consola y marca el estado en rojo; **no** puebla tablas ni C3D. Si la compilación es exitosa, llena la tabla de símbolos, la de tipos y el panel C3D, y reporta las rutas del `.c` y del ejecutable. La UI **no ejecuta** el binario.

10. Especificación de lenguajes

10.1 Lat (Pig Latin) — `.pig`

- Lenguaje de entrada; **sensible a mayúsculas**; puede importar `.y/.z`.
- No permite definir estructuras propias: se importan.
- Secciones: `VARIABLES>` (opcional), `MUNERA>` (opcional) y `MAIOR>` (obligatoria). Cierra con `FINIS;`.
- Variables globales y locales, funciones (`actio` sin retorno, `ratio` con retorno), condicionales (`si/aliter`), ciclos (`dum`, `facere`, `per`), `perge/interrumpe`, entrada/salida con `<<` y `>>`, objetos con `novus`.

Palabras y símbolos completos en el **Anexo C**.

10.2 Y? — `.y`

- **Sensible a mayúsculas**; la **indentación** define los bloques.
- Secciones `%estructuras` (opcional) y `%funciones` (obligatoria).
- No hay variables globales.
- Arreglos `[]` tipo nombre y estructuras `{}` Estructura nombre se pasan **por referencia**.
- Funciones definir `... -> tipo:`.
- Condicionales `si/sino/contrario`, `elegir/caso/siempre`, ciclos `para/mientras/hacer`, `romper/continuar`, `imprimir/leer`.

10.3 Zetariano — .z

- **Sensible a mayúsculas**; orientado a objetos.
- Un archivo = una clase; el nombre del archivo debe coincidir con el de la clase.
- Atributos públicos, constructores (sobrecargables) y métodos (sobrecargables).
- Objetos en el **heap**; new para objetos y arreglos; null.
- print/println/readln; ternario ? :; asignación compuesta += -= *;; switch/case/default; for/while/do.
- Sin herencia, encapsulamiento ni polimorfismo en esta entrega.

11. Tabla de compatibilidad de tipos

Resumen (la tabla completa, con todos los casos y ejemplos, está en el **Anexo B**):

- **Promoción numérica**: char → int → double.
- **Asignación**: solo se permite ensanchar. double → int es **error**.
- **+**: si algún operando es string, el resultado es string (concatenación); si no, aritmética promovida.
- **Relacionales**: solo numéricos.
- **Lógicos**: solo booleanos.
- **==/!=**: tipos comparables (mismo tipo, numéricos entre sí, clase vs null).
- **null**: assignable/comparable solo con clases (Z).

12. Diagrama de clases

El diagrama completo, con subdiagramas por capa en Mermaid y notas de las relaciones, está en el **Anexo D**.

13. Compilación y ejecución

13.1 Requisitos

- Java 17.
- Maven 3.9.x (no hay mvnw).
- gcc disponible en el PATH (para compilar el C generado).

13.2 Compilar el proyecto

`mvn compile` # o: `mvn -q compile`

13.3 Ejecutar (línea de comandos)

El punto de entrada activo es `com.navi.Main`, que recibe un archivo `.pig`, `.y` o `.z` (por defecto `testfiles/matriz_lat/main.pig`):

`mvn -q compile`

`CP="target/classes:$(mvn -q dependency:build-classpath -Dmdep.outputFile=/dev/stdout)"`

`java -cp "$CP" com.navi.Main testfiles/completo/main.pig`

El programa imprime la tabla de símbolos y las cuartetas, escribe `output/<nombre>.c`, limpia `output/` antes de cada corrida y compila con `gcc`.

13.4 Ejecutar la interfaz gráfica

Descomentar el bloque `main` alternativo en `src/main/java/com/navi/Main.java` (el que crea `CompilerWindow` y aplica `FlatMaterialDarkerIJTheme.setup()`), recompilar y ejecutar `com.navi.Main`.

13.5 Uso del `CompilerService` (embebido)

`CompilerService service = new CompilerService();`

`CompilationResult result = service.compile(Path.of("testfiles/completo/main.pig"));`

`if (result.isSuccessful()) {`

`String c3d = result.getC3d();`

`Path cFile = result.getCFile();`

`Path exe = result.getExecutable();`

`} else {`

`System.out.println(result.getMessage()); // reporte de errores`

`}`

14. Manejo de errores

Fase	Recolector	Formato	Efecto
Léxico/Sintáctico	CollectingErrorListener	[line:col] mensaje	SemanticError; aborta antes del AST
Semántico	SemanticErrors (en SemanticContext)	[line:col] mensaje	Se acumula; no se genera C3D
gcc	salida del proceso	texto de gcc	IllegalStateException

`SemanticReporter.report(ctx)` produce un texto con los errores, la tabla de tipos y la tabla de símbolos; es lo que imprime Main y lo que la UI vuelca en la consola.